



University of Vermont

THE UNIVERSITY OF VERMONT
COLLEGE OF ENGINEERING AND MATHEMATICAL SCIENCES
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
DEPARTMENT OF MECHANICAL ENGINEERING

User Manual

Team 485 - Autonomous Weeding Robot

ANDIE BROWN
NIELS KELLER
CADE LARCOM
JOHN POIRIER

Revision History

Date	Ver.	Description
04/29/2026	1.0	Initial Release

April 29, 2026

Contents

1	Mechanical	1
2	Software	2
2.1	Front tool	2
2.1.1	Instructions to Compile and Use	2
2.2	OptiTrack	3
2.2.1	Important Devices/Logins	4
2.2.2	OptiTrack Start Up Checklist	5
2.3	ROS2 Packages	8
2.3.1	demo_night	8
2.3.2	ml_red_controller	11
2.3.3	motor_tester	12
2.3.4	opti_waypoint_nav	12
2.3.5	simple_teleop	13

1 Mechanical

The multi-tool attachment system is designed to be easily accessible and easy to understand. The system uses a tongue and pin system for easy removal and placement of tools. To remove a tool, remove the ball pins from the tongue slots, and slide the tool off from the slots. To put a tool back on, simply slide the tool back onto the tongue slots, and replace the ball pins at each contact point. **Figure 1** shows this system up-close.

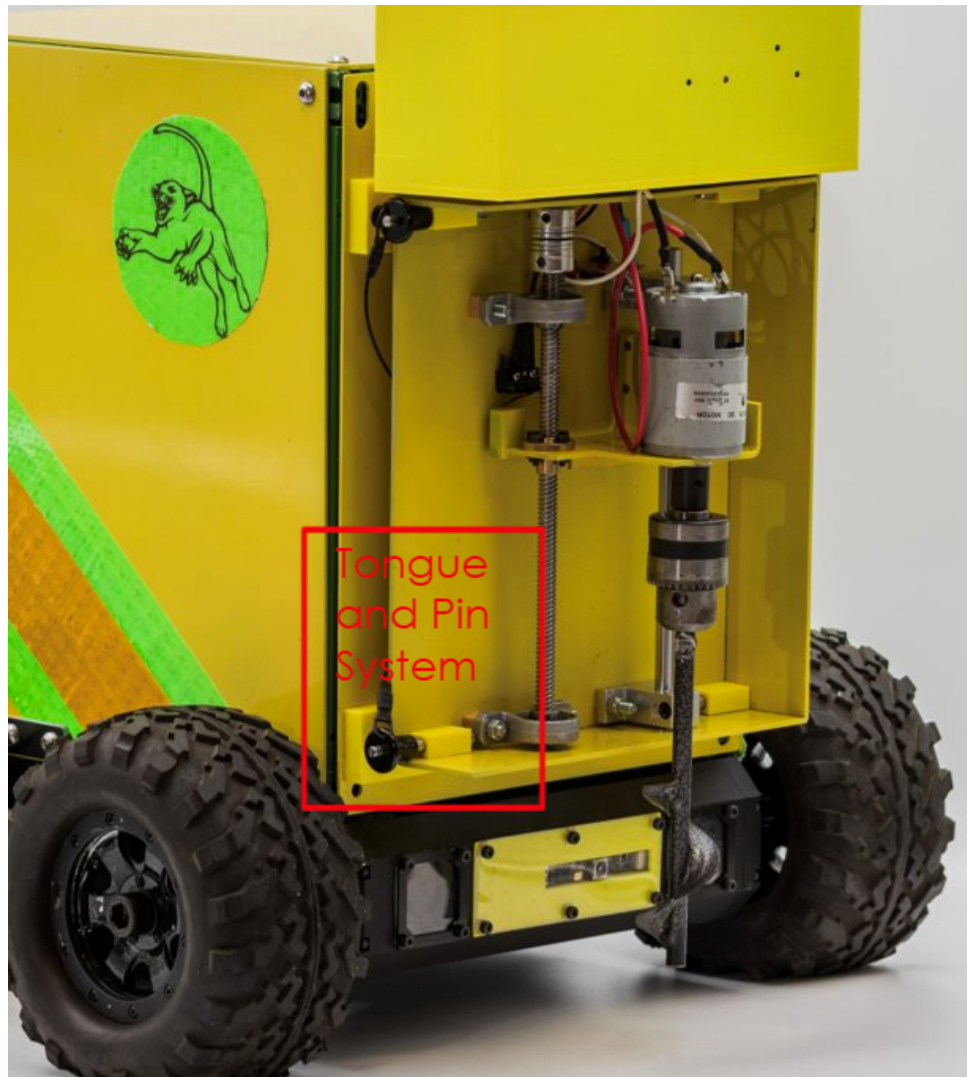


Figure 1. Quick Tool Attachment System

2 Software

2.1 Front tool

The following code applies to the ESP32 on the front tool board PCB, as well as the software to control it.

2.1.1 Instructions to Compile and Use

Use the Arduino IDE to open and upload the file of interest to the ESP32 via the micro USB port. There are two modes of operation for the front tool with corresponding firmware.

For testing/calibration purposes use the stepper terminal script. Connect a computer running the Arduino IDE. Make sure to select the port which the ESP32 is connected to, and select "DOIT ESP32 DEVKIT V1" as the board. It may be necessary to install the ESP32 boards via the board manager as shown in Fig 2.

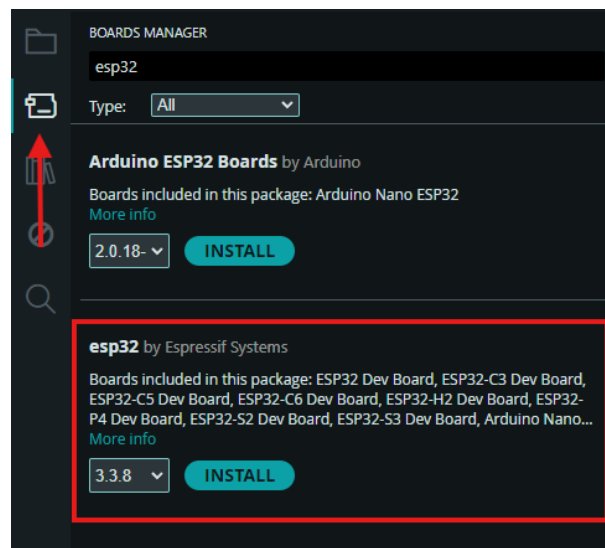


Figure 2. ESP32 Library in the board manager

Make sure that the line ending style is set to "New Line" as shown in Fig 3. Commands can be run from the terminal interface. Note that running the calibration scripts stores the results in the EEPROM. This means that when the main firmware is used the updated calibration constants will be used.

The other option is the front tool firmware. This is meant for direct communication with the NVIDIA Jetson. Note that this firmware is still a work in progress. Make sure to follow all safety guidelines while running tests with this script to avoid uncontrolled motor movements.

While both systems operate via sending serial commands from the host (Jetson or Computer) to the front tool microcontroller, the firmware does not have a text interface. The following commands are accepted by the microcontroller:

- "home\n"

- Finds the home position of the board. This must be run before anything else, but only once.
- "drill\n"
 - Runs the "doDrill" command. Note this command is still only partially implemented in this firmware. It is recommended to take the most recent version of doDrill from the stepper motor terminal program.
- "stop\n"
 - Stops the drill as soon as possible. Note, only rudimentary tested, it is recommended to thoroughly test this.

Note that "\n" is the newline escape sequence. This can be done automatically through the Arduino IDE using the serial monitor. Simply select "New Line" as the ending style as shown in Fig 3. This will insert "\n" at the end of your command.

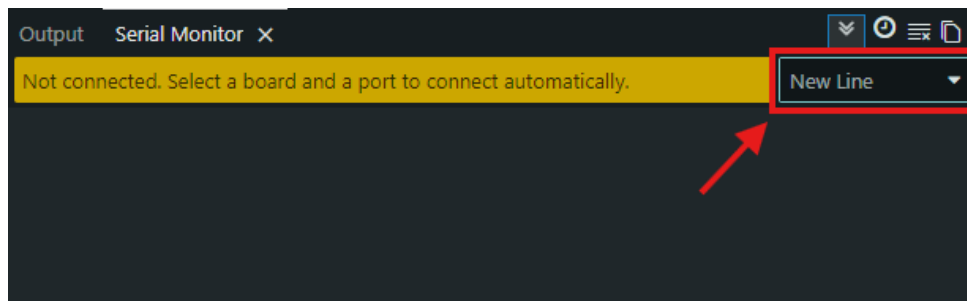


Figure 3. Automatic Arduino new line character setting.

Unlike the stepper terminal, the front tool firmware has a built in stop functionality. This is not included in the terminal script because all motor movements require constant inputs. This means that all (front tool) motor movements stop when either the 12 Vdc supply is turned off, or (more importantly) when the power to the microcontroller is turned off.

With both programs make sure to modify the parameters at the top of the file.

2.2 OptiTrack

OptiTrack is required for in lab use of the robot (when using odometry). It works by tracking the small spheres attached to the robot. These spheres are [retro-reflective](#), and are tracked by all 4 cameras. The OptiTrack system operates at 360 Hz and is accurate to under 1mm making it optimal for controls system testing.

The OptiTrack system diagram is shown in Fig. 4. The top 3 boxes are all for just the OptiTrack system itself whereas the bottom two are the added infrastructure in order to transmit the location data to the robot. The bottom two nodes communicate via the ROS2 domain ID protocol this works because the two devices are on the same network (the local Airlab network)

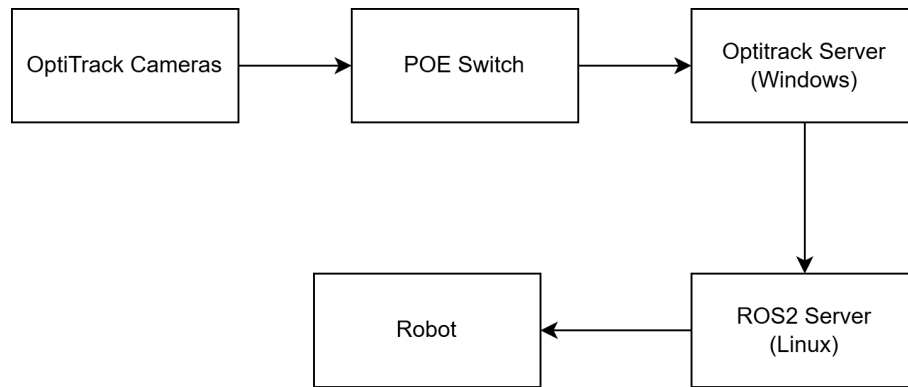


Figure 4. Optitrack Flowchart

2.2.1 Important Devices/Logins

- Robot Jetson:
 - Password: "airlab"
- ROS2 Server
 - Username: "AIRLAB"
 - Password: "cems46S3"
- Optitrack Server
 - No Password
- Airlab Wifi
 - SSID: UVMAirlab
 - Password: AIRLABV308

2.2.2 OptiTrack Start Up Checklist

1. Plug in OptiTrack power over Ethernet switch (Ethernet switch behind OptiTrack computer)
2. Power on all computers and log in with information from section 2.2.1
3. On the OptiTrack Server:
 - (a) Open up the software named "Motive"
 - (b) If OptiTrack was turned off, any of the camera towers have been bumped, or you are otherwise unsure of calibration [recalibrate them \(https://youtu.be/HyrHhaRV0aM?t=136\)](https://youtu.be/HyrHhaRV0aM?t=136). Check that the calibration is at least labeled as "Good" in the calibration review window. Finally Place the ground plane with the corner on the piece of tape in the center of the tracking space as shown in Fig 5.

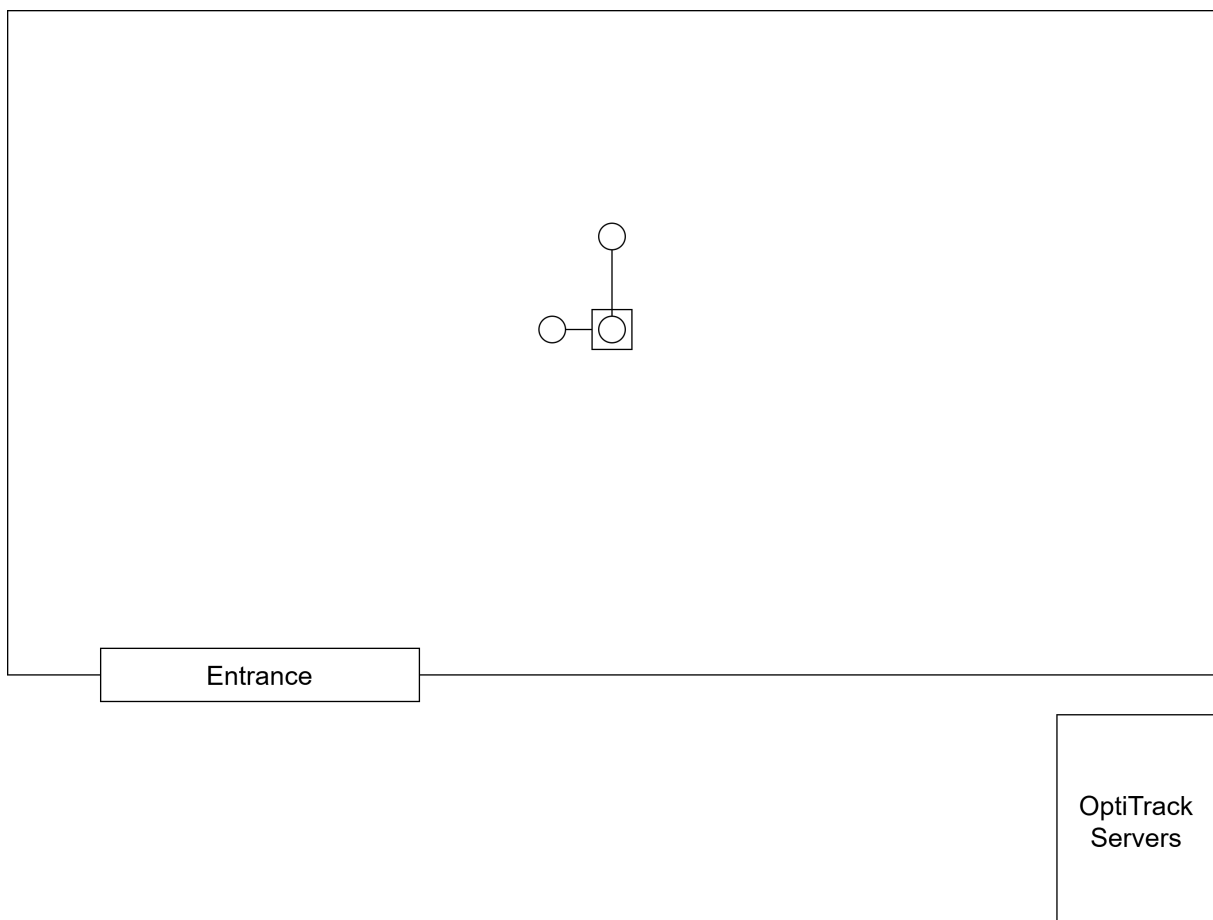


Figure 5. Ground Reference

4. Switch to the ROS 2 server:

NOTE: If viewing from A computer (instead of printed), better copy/paste commands can be found under the "Launch optitrack system:" section of:

https://github.com/MOCAP4ROS2-Project/mocap4ros2_optitrack/blob/rolling/README.md

- (a) (Optional, yet recommended) Open a terminal and run:

```
terminator
```

- (b) Right click and select split Vertical/Horizontal until you have 3 terminals. The recommended configuration is one wide terminal on top and two smaller ones below.

- (c) In the top wide terminal run:

```
ros2 launch mocap4r2_optitrack_driver optitrack2.launch.py
```

This is the main ROS2 node. You can press Ctrl + C at any time to quit, but then you must restart to this step.

- (d) In the first smaller window run:

```
ros2 lifecycle set /mocap4r2_optitrack_driver_node activate
```

This promotes the OptiTrack node to active.

- (e) (Optional) To launch the RQT viewer run:

```
rqt
```

- (f) (Optional) To launch RVIS run:

```
ros2 launch mocap4r2_marker_viz mocap4r2_marker_viz.launch.py mocap4r2_system
```

Troubleshooting steps can be found in [Table 1](#).

Symptom	Cause	Fix
rqt topic tracker shows “unknown”	mocap4Ros2 install/setup.bash is not sourced	1. Run <code>source ~/.bashrc</code> If still broken: 2. <code>cd</code> <code>/home/airlab/seed25/mocap4r2_ws/</code> 3. <code>source</code> <code>install/setup.bash</code>
No rigid bodies/markers in Ros Server RQT	Optitrack computer stopped streaming	Go to the Optitrack computer and open Motive. Edit → Settings → Streaming and enable applicable streaming.
When launching ROS2 server, system says “Trying to connect to Optitrack NatNET SDK at ...” but fails connection	Motive computer changed IP address or local interface mode	1. Change local interface to IP address: Motive → Edit → Settings → Streaming → Local Interface. Set it to the WLAN port (can be found with <code>ipconfig</code>). 2. If this fails, check the IP address of the Motive computer. If it changed, edit: <code>Seed25/mocap_ws/src/</code> <code>mocap4ros2_optitrack/</code> <code>mocap_optitrack\</code> <code>_driver/config/mocap\</code> <code>optitrack_driver\</code> <code>_params.yaml</code>

Table 1. Troubleshooting Steps

2.3 ROS2 Packages

ROS2 (Robot Operating System 2) packages serve as collections of scripts, called nodes, that can work together through the ROS communication structure. To utilize these packages, common practice is to create launch files, which allows any specified nodes of a package (or several packages) to be launched simultaneously via the single launch file. To operate the ROS packages on the Autonomous Weeding Robot, you must SSH into the NVIDIA Jetson from a personal machine. There are many ways to do this, choose your favorite. The Jetson has a displayport and four USB-A 3.2 ports that can be used to connect to a monitor, keyboard, and mouse, so that the network can be configured and the IP address for an SSH connect can be found. Note that the Jetson and the machine you will SSH from must be on the same network.

Setup and launch files are used to connect the nodes to the full packages. The `setup.py` file within packages calls the nodes within the packages so they can be executed via terminal commands. The `launch\launch.py` files within packages will launch all nodes defined in the launch file when the launch file is executed. Below documents the general steps of using a ROS package.

1. First, open the workspace that the packages are found in. This can be done using the `cd` command. For example, for the Jetson on the Autonomous Weeder Robot, use `cd seed25/ros_packages/seed_controller_ws` to get into the workspace that contains the packages.
2. Next, use `colcon build` to build all the packages in the workspace, or `colcon build --packages-select <PackageName>` to build specific packages. Optionally, you can instead use `colcon build --symlink-install` to build the packages, and allow changes in the node script file to update without having to re-build. Otherwise, you must re-run the build command any time the scripts are changed.
3. Then, use `source install/setup.bash` to source the workspace.
4. Finally, use the `ros2 run <PackageName> <NodeExecutableName>` command to run a single node or `ros2 launch` to run a launch file, allowing for multiple nodes to be run simultaneously. For example, to run the `teleop_node` node, use the command `ros2 run simple_teleop teleop_node`. To launch the `demo_night` package, use `ros2 launch demo_night launch.py`.
5. To terminate any running nodes, in the terminal that they were run, use **Ctrl+C** to close all active nodes from that terminal.

Note that you can also use the `cd ..` command to backtrack one directory level at any time. The following sections describe how to use specific ROS packages designed for the autonomous weeder robot.

2.3.1 demo_night

This package was designed as the all-in-one package for Design Night. The package functions, but is incredibly sensitive to environmental details (unwanted colors in the room, inconsistent lighting, unstable or high-latency network connection, etc.). The package opens a GUI that displays a dead-reckoning virtual twin, several options for robot operation mode, and a pop-up window for live view through the RealSense and Arducam cameras.

Before running this package, there are several calibration steps:

First, run a dead reckoning calibration. To do this, place the robot in a clear space where it can drive forward for four seconds. At least 20 feet of clear space straight ahead of the robot is recommended. Mark a reference point to be used for measuring distance after the test. Then, from the SSH home of the Jetson, run the following commands:

1. `cd seed25/ros_packages/seed_controller_ws/src/demo_night`
2. `python3 dead_reckoning_calibration.py`

This will have the robot run at full speed straight forward for four seconds. Then, once the robot has stopped, mark the end position based on the same reference point as before. Measure the distance, and input that distance into the terminal, in meters. The terminal will display that the data has been saved.

Second, it is helpful to calibrate the color masks of the cameras in new environments. To do so, follow these steps:

1. Place the robot in a clear, relatively empty space.
2. Run these commands from the SSH home of the Jetson:
 - (a) `cd seed25/ros_packages/seed_controller_ws/src/demo_night`
 - (b) `python3 realsense_color_sampler.py` or `python3 arducam_color_sampler.py`
3. Once the camera view pops up, place an object of the desired color in the view of the camera being used.
4. On the camera view window, click and drag a box that only includes the color you want to detect.
5. The terminal will output the average RGB, HSV, and recommended HSV range based on the selected area. You can change the HSV color mask ranges in the `gtg_controller_node.py` or `optical_path_follower_node.py` files (variables that begin with "UPPER" and "LOWER").

To run this package, use the following series of commands, starting from when an SSH terminal into the Jetson is just opened:

1. `cd seed25/ros_packages/seed_controller_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 launch demo_night launch.py`
5. Use **Ctrl+C** while in the terminal to terminate the package at any time.

Once the last command is run, the GUI will open, along with the camera view, as shown below in **Figure 6**. The figure is annotated, and the annotations (red numbers) correspond to the following:

1. IDLE/STOP - This is the robot idle mode. The robot will not do anything when idle.

2. REHOME SEQUENCE - This mode slowly spins the robot around, looking for ArUco codes used for homing the robot. Then, once it finds all four ArUco codes, it will attempt to center itself and orient itself to the north (ArUco code index 0) code.
3. OPTICAL PATH FOLLOWING - This enables the optical path following, looking for the blue tape as shown in the camera view and attempting to follow it indefinitely. Simultaneously, the go-to-goal mode is running, and if the robot detects red, it will divert from the optical path following to the red object using go-to-goal.
4. GO-TO-GOAL - This enables only the go-to-goal algorithm. If the robot detects red, it will navigate to the object. Once reaching the object, it will actuate the drill. The robot will not move if there is no detected object.
5. TELEOP - This is a simple remote manual operation mode. The user can control the robot by using the arrow keys displayed on the GUI (up/down for forward/backward, left/right for pivots).
6. REHOME DRILL - Before the drill can be actuated from the go-to-goal mode, it must home. The drill automatically homes on startup of the package, and may need to periodically be re-homed, which can be done via this button.
7. E-STOP DRILL - This button stops the drill at any point in case of emergency. Always verify this functionality before running critical tests or tests with safety concerns.
8. Information Display - This section displays the most recently requested mode of operation, the most recently confirmed mode of operation, what stage of the rehoming sequence the robot is in (if any), what stage of actuation the drill is in (if any), and the DR ratio calibrated from using the `dead_reckoning_calibration.py` script.

Additionally, the package launches the camera view, which displays both the RealSense and Arducam views, and any detected objects.

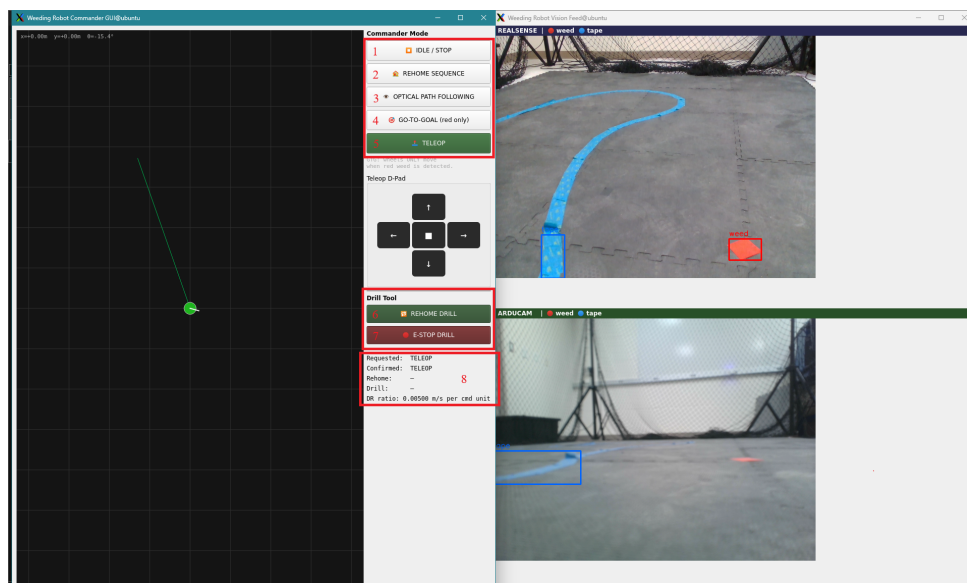


Figure 6. demo_night Package Screen Output

When running this package, there may be an error where the RealSense view window

displays that there is no output from the camera, and the Arducam view actually displays the RealSense view. To resolve this, go into the `arducam_node.py` script in the `demo_night` package and change the below line as needed. Commonly, the ID of the Arducam is either 0, 1, or 2. `self.declare_parameter('device_id', 0)`

2.3.2 ml_red_controller

This package was designed as the standalone package for pure go-to-goal using a machine learning model. For this package, the model is trained to detect small red rectangles. The package uses a bird's eye view transform on the live video feeds from the RealSense and Arducam, and then uses that video feed and the machine learning model to detect any test objects. Any test objects that are found are localized relative to the robot, and then a go-to-goal controller commands the robot toward the test object. To run this package, use the following series of commands, starting from when an SSH terminal into the Jetson is just opened:

1. `cd seed25/ros_packages/seed_controller_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 launch ml_red_controller launch.py`
5. Use **Ctrl+C** while in the terminal to terminate the package at any time.

Once the last command is run, the live camera feed will be displayed in a pop-up window that also displays when the robot detects a test object and the bird's eye view transformed live image. These displays are shown below in **Figure 7**.

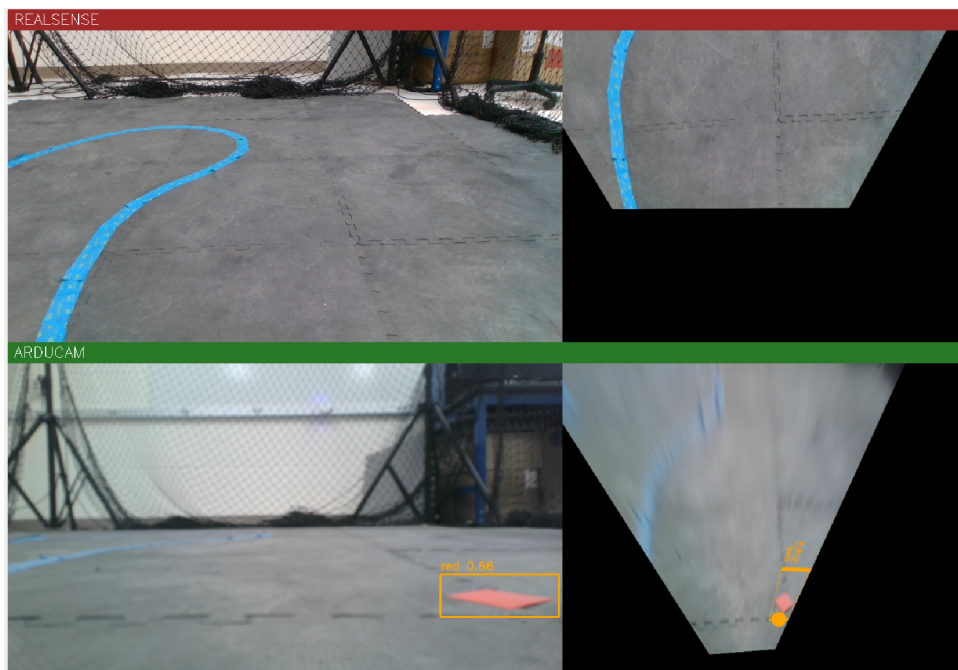


Figure 7. ml_red_controller Package Screen Output

2.3.3 motor_tester

This package was designed as a simple motor test to verify how to control the motors. It is a single-run package that goes through its cycle and then stops. This package does not wait for user input, so make sure that you are ready for the robot to move before running the node. The robot will pivot right, then pivot left, then move backward, then move forward, then do an arcing turn backward and left, then do an arcing turn backward and right, then do an arcing turn forward and left, and finally do an arcing turn forward and right. To run this test, use the following series of commands, starting when an SSH terminal into the Jetson is just opened:

1. `cd seed25/ros_packages/seed_controller_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 run motor_tester motor_test_node`
5. Use **Ctrl+C** while in the terminal to terminate the package at any time.

Once the last command is run, the motor test sequence will begin, and the robot will continue until it finishes all tests or the node is terminated. The terminal will output the current test that the robot is in within the sequence so the motor commands can be verified.

2.3.4 opti_waypoint_nav

This package was designed to be used with the OptiTrack system to simulate using RTK GPS on the robot. The package uses pre-coded waypoints, and then runs a PID path following controller to navigate through all the waypoints sequentially. Robot position information is gathered from OptiTrack. To set up OptiTrack, see section [2.2](#).

Once the OptiTrack system is streaming the rigid body data of the robot, run the following series of commands, starting when an SSH terminal into the Jetson is just opened:

1. `cd seed25/ros_packages/seed_controller_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 launch opti_waypoint_nav launch.py`
5. In a second terminal, after the package has had time to launch, run this command: `ros2 topic pub --once /start_mission std_msgs/msg/Empty {}` - You may need to run this line multiple times for the package to actually receive the message.
6. Use **Ctrl+C** while in the terminal that the launch file was run to terminate the package at any time.

Once the package is launched, a virtual twin window and a camera view window will open. These windows can be seen below in **Figure 8**. Once the running package receives the message from the second terminal, the robot will begin the waypoint path, and will continue until the robot finishes the path or the package is terminated.

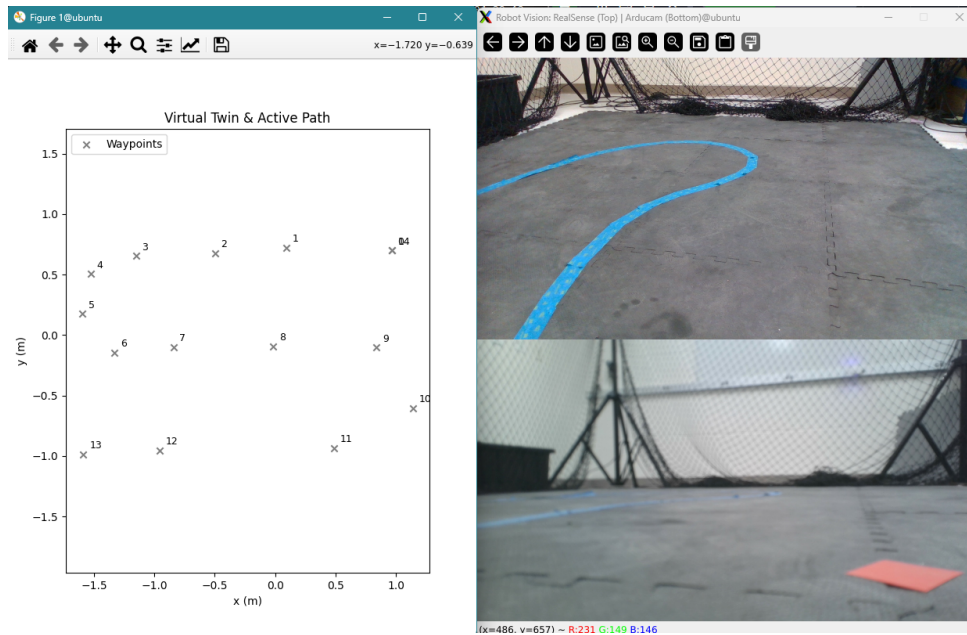


Figure 8. opti_waypoint_nav Package Screen Output

2.3.5 simple_teleop

This package was designed to be used for use between tests, making it easier to move the robot around instead of picking it up and moving it. This package did not satisfy any customer requirements, and was purely for convenience during development. To run this package, use the following series of commands, starting when an SSH terminal into the Jetson is just opened:

1. `cd seed25/ros_packages/seed_controller_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 run simple_teleop teleop_node`
5. Use **Ctrl+C** while in the terminal to terminate the package at any time.

Once the node is launched, the terminal will output instructions. The up arrow key can be used to move forward, the back arrow key to move backward, the left arrow key to pivot left (counter-clockwise), and the right arrow key to pivot right (clockwise). Due to the trapezoidal motor profile, it takes a moment for the robot to actually start moving when an arrow key is held down, and the robot cannot process multiple arrow keys simultaneously.

This script also serves as the most convenient test to ensure the motors are functioning properly. If you run this script and the motors will not turn, there are two common errors. First, ensure that the motor command wires are plugged into the Jetson properly, as shown in **Figure 9**. The other common issue is that the breaker on the internal wall opposite the Jetson needs to be reset. Turn off the robot, reset the breaker, and try again.

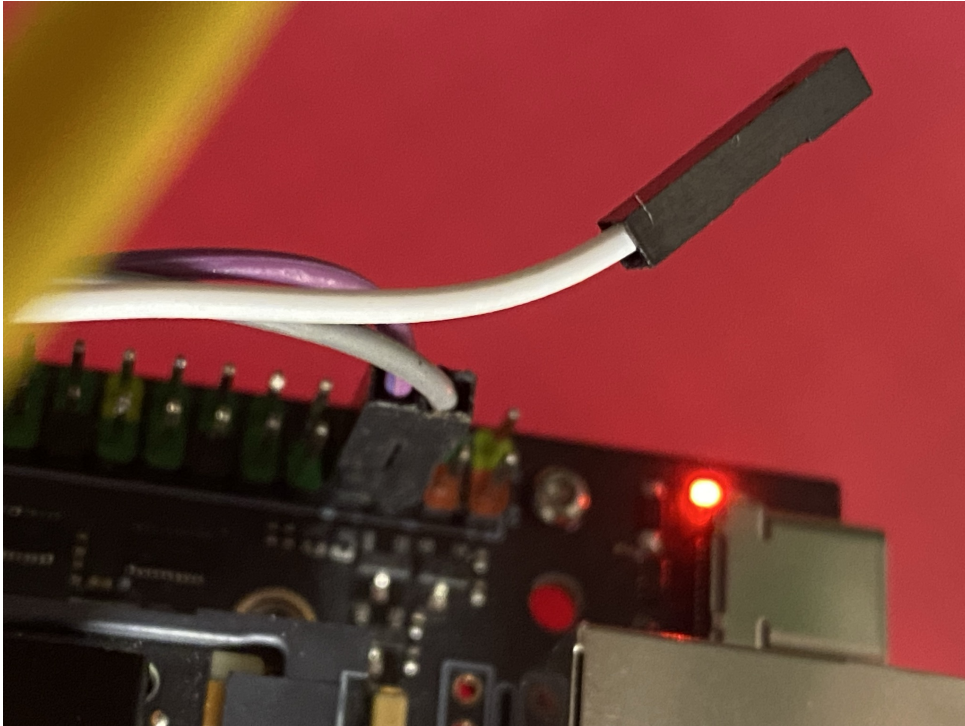


Figure 9. Motor Command Wire Connections to the NVIDIA Jetson